TODO operativo - Update sicuro di input_odp

Checklist differita ma pronta all'esecuzione per evolvere il sync ODP senza sovrascrivere stato operativo, fasi attive e progressi locali.

Ambito **sync_input.py, input_odp, routes.py, odp_in_carico**	**Obiettivo** **Introdurre un update/upsert sicuro dei dati ERP**	**Vincolo** **Nessun reset di StatoOrdine, FaseAttiva o progressi locali**
Uso previsto Documento da seguire in un secondo momento	**Formato** Human-readable + AI-friendly	**Esito atteso** Piano implementabile per fasi, con criteri di accettazione

Formato consigliato per TODO tecnici riusabili anche da AI: manifesto iniziale, backlog con ID stabili, task card con dipendenze e definition of done, checklist finale e appendice machine-readable. Questo PDF usa esattamente questa struttura.

1. Executive summary

L'architettura attuale usa input_odp sia come snapshot dell'ERP sia come sorgente letta dalla UI per rappresentare lo stato operativo. Questa doppia responsabilità è gestibile finché il sync inserisce solo nuove righe; diventa rischiosa nel momento in cui si introduce un vero update/upsert.

- sync_input.py oggi filtra le PK già presenti e inserisce solo righe nuove; questo riduce il rischio immediato ma impedisce la correzione strutturale dei record già caricati.
- Le route operative modificano StatoOrdine e usano anche FaseAttiva; quindi un update cieco di input_odp può riportare ordini lavorati a uno stato incoerente.
- odp_in_carico contiene già il runtime locale utile a separare esecuzione e snapshot ERP: Stato_odp, Data_in_carico, Tempo_funzionamento, Utente_operazione, Fase, data_ultima_attivazione.
- La soluzione corretta non è un semplice UPDATE su input_odp, ma una separazione esplicita tra campi governati dall'ERP e campi governati localmente.

2. Invarianti da rispettare

- Il sync ERP non deve mai resettare Attivo/In Sospeso/Pianificata se quel valore è usato come stato operativo locale.
- La quantità originale ERP non va riscritta con i residui di produzione; per chiusure parziali servono campi o tabelle dedicate al progress locale.
- La fase attiva mostrata in UI deve dipendere dallo stato runtime o da un calcolo server-side, non da una semplice riscrittura indiscriminata di input_odp.
- DistintaMateriale deve essere JSON valido, senza token NaN e senza righe vuote di fase.
- Qualsiasi update deve essere idempotente: due sync consecutivi con stessi dati ERP non devono cambiare il runtime locale.

3. Mappa dei dati e ownership

Oggetto	Owner logico	Scopo	Aggiornabile dal sync?	Nota
input_odp	ERP snapshot	Anagrafica ordine + pianificazione	Si, ma solo campi ERP	Evitare update cieco di StatoOrdine/Fase Attiva
odp_in_carico	Runtime locale	Stato effettivo di lavorazione	No	Fonte migliore per Attivo/In Sospeso/tempo
change_event	App locale	Eventi live / bridge frontend	No	Non va rigenerata dal sync storico
*_log	Audit locale	Storico e analisi	No	Append-only
Futura odp_progress	Runtime locale	Prodotto/scarto/ residuo per parziali	No	Serve prima delle chiusure parziali

4. Roadmap sintetica

Fase	Obiettivo	Output	Rischio se saltata	Priorita
F0	Mappare campi ERP vs locali	Documento di ownership	Upsert pericoloso	Alta
F1	Sanificare payload del sync	JSON pulito e stabile	Errori frontend / parsing	Alta
F2	Progettare update selettivo	Lista campi aggiornabili	Reset stati e fasi	Alta
F3	Introdurre progress locale	Tabella residui/parziali	Parziali non affidabili	Alta
F4	Derivare stato UI server-side	Home robuste	Divergenza tra tabelle	Media
F5	Test e rollout protetto	Suite regressione + fallback	Errori in produzione	Alta

5. Backlog operativo con ID stabili

ID	Titolo	Outcome atteso
T01	Inventario dei campi	Classificare ogni colonna di input_odp come ERP-owned, local-owned o derivata.
T02	Specifica invarianti	Scrivere le regole che il sync non deve mai violare.
T03	Sanificazione DistintaMateriale	Eliminare righe vuote/NaN e serializzare solo JSON valido.
T04	Design update selettivo	Definire elenco dei campi di input_odp aggiornabili da ERP.
T05	Design progress locali	Definire tabella per prodotto, scarto, residuo, chiusure parziali.
T06	Stato calcolato server-side	Derivare stato effettivo da odp_in_carico + eventuale progress.
T07	Refactor sync	Introdurre INSERT per nuovi ordini e UPDATE controllato solo sui campi ERP.
T08	Compatibilita route	Adeguare apertura, sospensione, riattivazione, chiusura totale/parziale.
T09	Migrazione dati	Strategia per ordini attivi già presenti e dati sporchi pregressi.
T10	Test di regressione	Casi macchina, SL, lotti, matricole, parziali, re-sync, replay.
T11	Feature flag / rollout	Attivazione progressiva con fallback.
T12	Osservabilita	Log chiari e controlli post-sync.

T01 - Inventario dei campi e ownership

Scopo	Bloccare a documento quali campi sono sotto controllo ERP e quali sono locali o derivati.
File/oggetti	models.py (InputOdp, StatoOdp), routes.py, sync_input.py
Dipendenze	Nessuna
Rischio principale	Un campo locale viene classificato come ERP e poi sovrascritto durante il refactor.

- Estrarre tutte le colonne di input_odp e segnare owner = ERP / locale / derivato.

- Evidenziare almeno: StatoOrdine, FaseAttiva, DistintaMateriale, Quantita, DataInizioSched, DataFineSched, CodRisorsaProd.
- Verificare se alcune colonne locali oggi sono scritte solo dalle route ma lette anche dal sync o dalla UI.

Definition of done: Esiste una tabella di ownership approvata e usata come base per tutti i task successivi.

T02 - Specifica invarianti e casi proibiti

Scopo	Tradurre la logica di business in regole che ogni update deve rispettare.
File/oggetti	Documento tecnico + test plan
Dipendenze	T01
Rischio principale	L'update viene progettato senza coprire i casi reali di produzione.

- Scrivere esempi concreti: ordine Attivo, In Sospeso, chiusura totale, futura chiusura parziale, ordine macchina con matricola, ordine SL con lotti.
- Definire stati e transizioni vietate: es. Attivo -> Pianificata per effetto del solo sync.
- Stabilire quali campi possono cambiare durante un re-sync e quali no.

Definition of done: Ogni scenario critico ha una regola esplicita e verificabile.

T03 - Sanificazione della DistintaMateriale

Scopo	Garantire che DistintaMateriale sia sempre JSON valido e semanticamente pulito.
File/oggetti	sync_input.py, eventuali utility di serializzazione
Dipendenze	T01
Rischio principale	Persistenza di JSON sporco che rompe frontend o route di validazione.

- Convertire NaN/NaT in null o scartare le righe prive di CodArt reale.
- Mantenere eventuale NumFase ma non salvare pseudo-righe vuote della fase.
- Imporre serializzazione con allow_nan=False e test dedicato.

Definition of done: Qualsiasi record salvato nel DB è parsabile sia da Python che dal browser senza fallback speciali.

T04 - Design dell'update selettivo su input_odp

Scopo	Introdurre un meccanismo di update solo per campi ERP, senza toccare runtime locale.
File/oggetti	sync_input.py, eventuale funzione upsert dedicata
Dipendenze	T01, T02, T03
Rischio principale	Upsert cieco con reset di stato o fase.

- Creare una whitelist esplicita dei campi aggiornabili da ERP.
- Eseguire UPDATE solo se il record esiste e solo sui campi in whitelist.

- Conservare inserimento di nuovi ordini come flow separato.
- Evitare side effect su StatoOrdine, FaseAttiva e note locali finché la separazione non è completa.

Definition of done: L'ordine già presente può ricevere aggiornamenti anagrafici senza alterare lo stato di esecuzione locale.

T05 - Modello per chiusure parziali e residui

Scopo	Preparare la struttura dati locale per prodotto/scarto/residuo senza riutilizzare Quantita originale ERP.
File/oggetti	Nuova tabella (es. odp_progress), route di chiusura, log
Dipendenze	T02
Rischio principale	Residui e parziali sovrascrivono il dato originario, rendendo il sync ambiguo.

- Definire chiave logica: IdDocumento, IdRiga, Fase, eventuale progressivo evento.
- Salvare QuantitaOrdineOriginale, QuantitaProdottaTot, QuantitaScartataTot, QuantitaResidua.
- Separare il dato di pianificazione ERP dal consuntivo operativo.

Definition of done: Le chiusure parziali aggiornano solo la struttura locale dedicata e non cambiano la quantità ERP di base.

T06 - Stato UI derivato server-side

Scopo	Fare in modo che le home leggano uno stato calcolato e non il solo valore raw di input_odp.
File/oggetti	routes.py, home.j2, template home_*
Dipendenze	T01, T02, T05
Rischio principale	Frontend che continua a interpretare un campo ambiguo.

- Derivare lo stato effettivo da odp_in_carico, eventuale progress locale e stato ERP solo come fallback.
- Esporre al frontend uno stato coerente e già risolto server-side.
- Ridurre la dipendenza della UI da StatoOrdine scritto direttamente su input_odp.

Definition of done: Le tabelle Da eseguire / In corso restano coerenti anche dopo re-sync o rollout parziale.

T07 - Refactor del sync

Scopo	Applicare la nuova logica in produzione con insert di nuovi ordini e update selettivo per quelli esistenti.
File/oggetti	sync_input.py, query helper, logging
Dipendenze	T03, T04
Rischio principale	Comportamento non deterministico su cicli successivi.

- Separare pipeline: prepare payload ERP -> compare -> insert new -> update existing allowed fields.
- Loggare quanti record sono nuovi, quanti aggiornati, quanti ignorati e perché.
- Rendere l'operazione idempotente.

Definition of done: Due esecuzioni consecutive con stessi dati producono zero effetti collaterali sul runtime locale.

T08 - Compatibilita delle route operative

Scopo	Allineare apertura, sospensione, riattivazione e chiusura alla nuova separazione dei dati.
File/oggetti	routes.py, eventuali helper shared
Dipendenze	T05, T06, T07
Rischio principale	Stato duplicato in più tabelle con divergenze difficili da tracciare.

- Rimuovere, dove possibile, la necessità di usare input_odp come storage dello stato runtime.
- Far sì che chiusura totale e futura parziale usino dati locali dedicati per residui e tempi.
- Verificare anche ordini macchina con GestioneMatricola e componenti con GestioneLotto.

Definition of done: Le route continuano a funzionare sia nel pre-rollout sia nel post-rollout.

T09 - Migrazione dati e recupero storico

Scopo	Definire come trattare gli ordini già attivi e i record pregressi con dati incompleti o sporchi.
File/oggetti	Script SQL/Python dedicato, DB locale
Dipendenze	T03, T07
Rischio principale	Perdita o corruzione del runtime degli ordini già in lavorazione.

- Identificare record con DistintaMateriale sporca o campi locali già valorizzati.
- Stabilire se migrare, congelare o ricalcolare alcuni campi.
- Preparare script reversibile e report pre/post migrazione.

Definition of done: Esiste una procedura di migrazione con backup, report e criterio di rollback.

T10 - Test di regressione

Scopo	Bloccare i comportamenti attesi prima del rollout.
File/oggetti	pytest, fixture SQLite, eventuali casi integrati frontend/backend
Dipendenze	T02, T03, T05, T07, T08
Rischio principale	Refactor corretto sulla carta ma non protetto da regressioni.

- Copertura minima: nuovo ordine, re-sync stesso ordine, ordine Attivo, ordine In Sospeso, chiusura totale, parziale, ordine macchina, ordine SL, lotti/matricole, distinta vuota di fase.
- Verificare che il sync non riporti stato o fase a valori iniziali quando esiste runtime locale.
- Aggiungere test espliciti su JSON DistintaMateriale valido.

Definition of done: La suite fallisce se un re-sync altera stato, fase, tempi o progressi locali.

T11 - Feature flag e rollout

Scopo	Attivare la nuova logica in modo graduale.
File/oggetti	Config TOML / env / codice sync
Dipendenze	T07, T10

Rischio principale	Rollout tutto in una volta con ripristino difficile.

- Introdurre flag per tenere separati vecchio e nuovo comportamento.
- Attivare inizialmente solo in ambiente di test o su sottoinsieme controllato.
- Mantenere opzione di fallback immediato.

Definition of done: La nuova logica può essere disattivata senza migrazioni distruttive.

T12 - Osservabilita e controlli post-sync

Scopo	**Vedere subito se il sistema entra in stato incoerente dopo un update.**
File/oggetti	logging, dashboard, query diagnostiche
Dipendenze	T07
Rischio principale	Bug silenziosi che emergono solo dopo giorni.

- Loggare mismatch tra input_odp e stato derivato.
- Preparare query diagnostiche: ordini con runtime locale ma stato ERP divergente, ordini con residuo negativo, ordini con distinta non parsabile.
- Definire un controllo giornaliero o manuale post-rilascio.

Definition of done: Esistono segnali rapidi per individuare errori senza aspettare sintomi in UI.

6. Ordine consigliato di esecuzione

- Prima tranche minima: T01 -> T02 -> T03 -> T04 -> T10.
- Tranche strutturale: T05 -> T06 -> T07 -> T08 -> T11 -> T12.
- T09 va pianificato solo quando il nuovo flow è già stabilizzato in test.

7. Checklist finale pre-implementazione

Check	Descrizione	Esito	Note
C01	Tabella di ownership approvata	[ ]	
C02	Invarianti documentate	[ ]	
C03	JSON DistintaMateriale pulito	[ ]	
C04	Whitelist campi ERP definita	[ ]	
C05	Design progress parziali definito	[ ]	
C06	Strategia UI server-side definita	[ ]	
C07	Suite test minima pronta	[ ]	
C08	Piano di rollout e fallback pronto	[ ]	

Appendice A - Blocco AI-friendly

Questa appendice è pensata per essere copiata in un'altra chat o in uno strumento automatico. Usa ID stabili, vincoli espliciti e output richiesti.

```
project: input_odp_safe_update
goal: >
  Introdurre update/upsert di input_odp senza sovrascrivere stato operativo locale,
  fase attiva, tempi di lavorazione o progressi di chiusura parziale.
scope:
  - sync_input.py
  - models.py
  - routes.py
  - input_odp
  - odp_in_carico
future_entities:
  - odp_progress
hard_constraints:
  - Non resettare Attivo/In Sospeso/Pianificata per effetto del solo sync
  - Non sovrascrivere la quantità ERP originale con residui locali
  - DistintaMateriale deve essere JSON valido senza NaN
  - Due sync identici devono essere idempotenti
source_of_truth:
  input_odp: snapshot ERP
  odp_in_carico: runtime locale
  odp_progress: progress locale per parziali
tasks:
  - id: T01
    title: Inventario campi e ownership
    deliverable: tabella ERP/local/derived per ogni colonna
  - id: T02
    title: Specifica invarianti
    deliverable: elenco transizioni permesse e vietate
  - id: T03
    title: Sanificazione DistintaMateriale
    deliverable: serializer JSON pulito + test
  - id: T04
    title: Design update selettivo
    deliverable: whitelist campi aggiornabili
  - id: T05
    title: Design progress locali
    deliverable: schema tabella per parziali e residui
  - id: T06
    title: Stato UI derivato server-side
    deliverable: funzione/stato esposto coerente
  - id: T07
    title: Refactor sync
    deliverable: insert nuovi + update controllato esistenti
  - id: T08
    title: Compatibilita route
    deliverable: apertura/sospensione/riattivazione/chiusura allineate
  - id: T09
    title: Migrazione dati
```

```
      deliverable: piano backup + rollback + script
    - id: T10
      title: Test regressione
      deliverable: suite minima con casi reali
    - id: T11
      title: Feature flag
      deliverable: rollout controllato e fallback
    - id: T12
      title: Osservabilita
      deliverable: log e query diagnostiche
  acceptance:
    - Un re-sync non altera stato runtime locale
    - Un re-sync non altera fase attiva locale
    - Le chiusure parziali non cambiano la quantità ERP originale
    - Le home restano coerenti dopo il sync
```

Appendice B - Prompt operativo riutilizzabile

Usa questo backlog come piano di lavoro. Procedi un task per volta, senza introdurre update ciechi su input_odp. Per ogni task restituisci: analisi dello stato attuale, file da toccare, patch proposta, rischi, test minimi e criterio di rollback. Non modificare campi locali finché non esiste una tabella di ownership approvata.